

Análise de desempenho de algoritmos de ordenação para ambiente sob estresse

Emanuel Rauter, Geancarlo Kozenieski, Tiago Elias Steinke

Comparação entre algoritmos de Sorting em tempo, memória e potência para diferentes níveis de stress de CPU e memória.

Algoritmos com $O(n \log n)$:

Quicksort: Algoritmo de divisão e conquista que utiliza pivô, tem tempo médio de $n \log n$, porém em casos extremos pode cair para n^2 o tempo. Utiliza $\log n$ de espaço.

Mergesort: Algoritmo de divisão e conquista que divide vetor em 2, tem tempo médio de $n \log n$ para qualquer caso, porém utiliza n de espaço.

Heapsort: Utiliza a árvore binária para ordenar vetor. Tem tempo médio de $n \log n$ para qualquer instância e uso de espaço 1.

Justificativa: Algoritmos de sorting fazem parte do dia-a-dia do programador, e apesar de saber que os três algoritmos a serem analisados possuem mesmo tempo assintótico médio ($n \log n$), o grupo se interessou em verificar mais intensamente o tempo de execução, uso de memória e potência elétrica utilizada na sua execução. Além disso, o grupo quer verificar como estes algoritmos escalam quando aplicados estresse.

Método escolhido: Como será medido o uso de energia do hardware físico em tempo real, foi escolhido o método de medição. Será utilizado ambiente Linux pois facilita o acesso aos contadores e dados de performance além de preservar maior estabilidade do sistema.

Ambiente: Laptop AORUS 15P XD 11th Gen Intel i7-11800H 2 GB Ddr4 3200MHz

Sistema: Ubuntu 24.04.3 6.17.0-23-generic

Experimento: Ordenar diferentes vetores de tamanho n sob condição de estresse x .

Condições de estresse: 3 condições de estresse, 3 cargas de estresse

-Condição ambiente (sem estresse extra): Caso sem estresse extra para verificar funcionamento em condições normais

-Estresse CPU: Caso com estresse em CPU.

-Estresse RAM: Caso com estresse em RAM

-Estresse both: Caso com estresse em CPU e em RAM

-Carga 0%: Sem estresse

-Carga 50%: 50% da carga

-Carga 100%: 100% da carga aplicada

Entradas: 8 vetores na base 10^7

Justificativa: Entradas grandes o suficientes para ter tempo de execução na casa de segundos (impede que fatores aleatórios não controláveis do computador interfiram nos testes e medidas), sem serem longos demais para tornar a testagem inviável.

Estrutura de vetores:

- 1- 1 vetor ordenado
- 2- 1 vetor na ordem inversa
- 3- 1 vetor aleatório
- 4- 1 vetor de números repetidos
- 5- 1 vetor de vários números repetidos (1 a 5)

Análise de desempenho de algoritmos de ordenação para ambiente sob estresse

Emanuel Rauter, Geancarlo Kozenieski, Tiago Elias Steinke

6- 1 vetor zig-zag (0, Int-max, 1, Int-max -1, 2...)

7- 1 vetor ordenado com ruído no final (1%)

8- 1 vetor ordenado com swaps aleatórios

A geração do vetor não será incluída nos cálculos de tempo, memória e energia.

Ferramentas:

Linguagem dos algoritmos: C++, linguagem de mais baixo nível e que não possui garbage collector, resultando em dados menos poluídos.

Análise dos dados: Python, pois contém as melhores bibliotecas para análise dos dados coletados. Bibliotecas pandas e numpy para análise numérica e matplotlib para criação de gráficos.

Estresse: Criação de threads de estresse dentro do código.

Métricas medidas:

Tempo - Duração do início do algoritmo até a entrega do vetor totalmente ordenado:

Medição dentro do próprio programa por bibliotecas como chrono.

Uso de memória - Quantidade de memória RAM utilizada durante o processo:

Monitoramento RSS (Resident Set Size) do Linux através de funções proc e pid.

Memória pico e memória média medidas.

Custo energético - Monitoramento por RAPL no Linux (somente processadores Intel e AMD)

Delineamento do experimento:

Uso de fatorial completo:

- 8 vetores
- 7 níveis de estresse
- 3 algoritmos
- 5 repetições de cada combinação

Total de execuções:

$$8*7*3*5 = 840 \text{ execuções.}$$

5 repetições permitem mitigar variabilidades do sistema e permitem calcular média, desvio padrão e intervalos de confiança para cada métrica.